Project Name: 2048

This is a web-based implementation of the 2048 game. The objective of the game is to merge tiles with the same values to reach the 2048 tile. The application includes a responsive user interface, automatic theme switching based on system settings (light/dark mode), and Progressive Web App (PWA) support.

Role A – UI: Lazurko Danylo

Responsible for the user interface, styling, PWA, responsive design, theming and overall user experience.

Role B – Logic: Luděk Vecsey

Responsible for implementing the game logic, including tile movement, merging mechanics, score calculation, and game state management.

Repository:

https://github.com/danilmimozyra/2048

Application Goal

The goal of this application is to provide a simple and accessible version of the 2048 game that can be played in a web browser. The game is designed for anyone with intent of spending a few minutes playing a logical puzzle.

The application solves the problem of responsive design by adapting to every device and a problem of accessibility. Users do not need to install anything from an app store, as the game can be opened instantly in a browser and even installed on their device if they choose to.

Used Technologies

- React
- TypeScript
- Vite
- CSS
- Git & GitHub
- GitHub Actions
- GitHub Pages
- Vite Plugin PWA
- ChatGPT AI
- Claude AI

Architecture and Component Structure

Components

App

The root component of the application. It manages the global state, including animation phase and theme. It connects all parts of the application together and passes data to child components.

GameHeader

Displays the title, current score, best score, and a button to start a new game.

Props:

- *score* – current score
- *bestScore* – highest score
- *newGameCallback* – function to reset the game

GameBoard

Represents the game board and handles user interaction. It processes keyboard and swipe input and renders the grid and all tiles. It also adjusts tile states based on the animation phase.

Props:

- *tiles* – array of tile objects
- *animPhase* – current animation phase
- *move* – function for handling movement
- *tileColors* – array of colors for tiles

TileView

Displays a single tile. It calculates its position and visual appearance based on its properties.

Props:

- *tile* – tile data (position and value)
- *specialClass* – visual state of the tile
- *tileColors* – color palette
- *posMultiplier* – value used for positioning

Custom Hooks

useGame

Handles the main game logic and state management. It manages tiles, score, animations, and game reset. It also controls animation phases and movement timing.

useKeyboard

Listens for keyboard input and triggers movement based on pressed keys.

useSwipe

Handles touch and mouse swipe gestures and converts them into movement directions.

useTheme

Detects the system theme (light/dark) and updates CSS variables accordingly.

useTileSizeGap

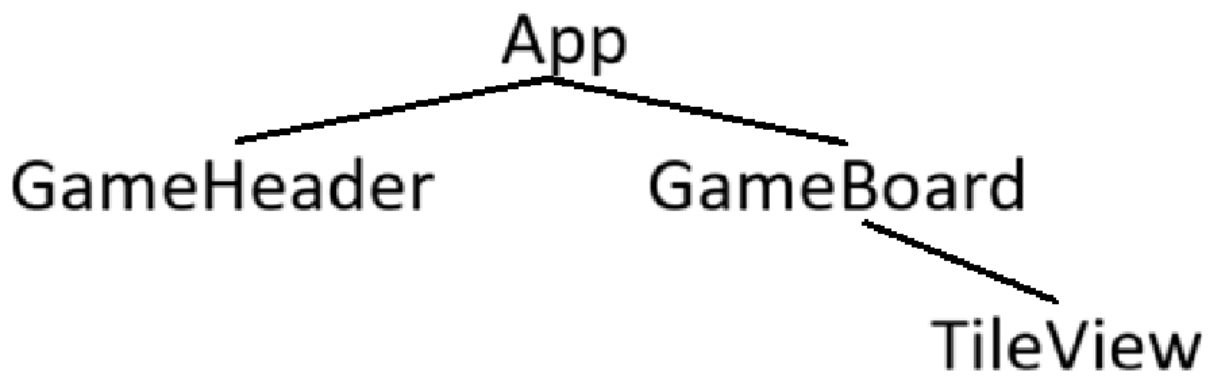
Reads CSS variables and provides tile size and spacing for positioning.

Data Flow

The main game state is created and managed inside the ***useGame*** hook, which is used in the ***App*** component.

1. The App component receives tiles, score, and control functions from ***useGame***.
2. These values are passed down to child components via props.
3. ***GameBoard*** renders tiles and passes each tile to ***TileView***.
4. User input (keyboard or swipe) is captured in ***GameBoard*** and handled by custom hooks.
5. Movement actions are passed back to ***App*** through the move function.
6. The game logic updates tiles and score, which causes the UI to re-render.

Component Tree



State Management

The main game state is managed using React state inside the ***useGame*** hook. The following data is stored in state:

- *tiles* – array of tile objects (position, value, state)
- *score* – current game score
- *animPhase* – current animation phase (idle, moving, spawning)

No data is stored in *localStorage* or *sessionStorage*. All state exists only during the current session.

Action Flow

When the user performs an action (keyboard press or swipe):

1. The input is detected by ***useKeyboard*** or ***useSwipe***.
2. The *move(direction)* function is called.
3. Inside ***useGame***, the move function updates the tiles using the game logic (*game.ts*).
4. The animation phase is updated (moving → spawning → idle).
5. React updates the state (tiles, score).
6. The UI is re-rendered:
    - ***GameBoard*** updates tile positions
    - ***TileView*** updates tile appearance

Deployment

The application is deployed using GitHub Pages. To build the project, the following command is used:

npm run build

Deployment is handled automatically using GitHub Actions. When changes are pushed to the main branch, the workflow installs dependencies, builds the project and deploys it to GitHub Pages

The application is publicly available at: https://danilmimozyra.github.io/2048/

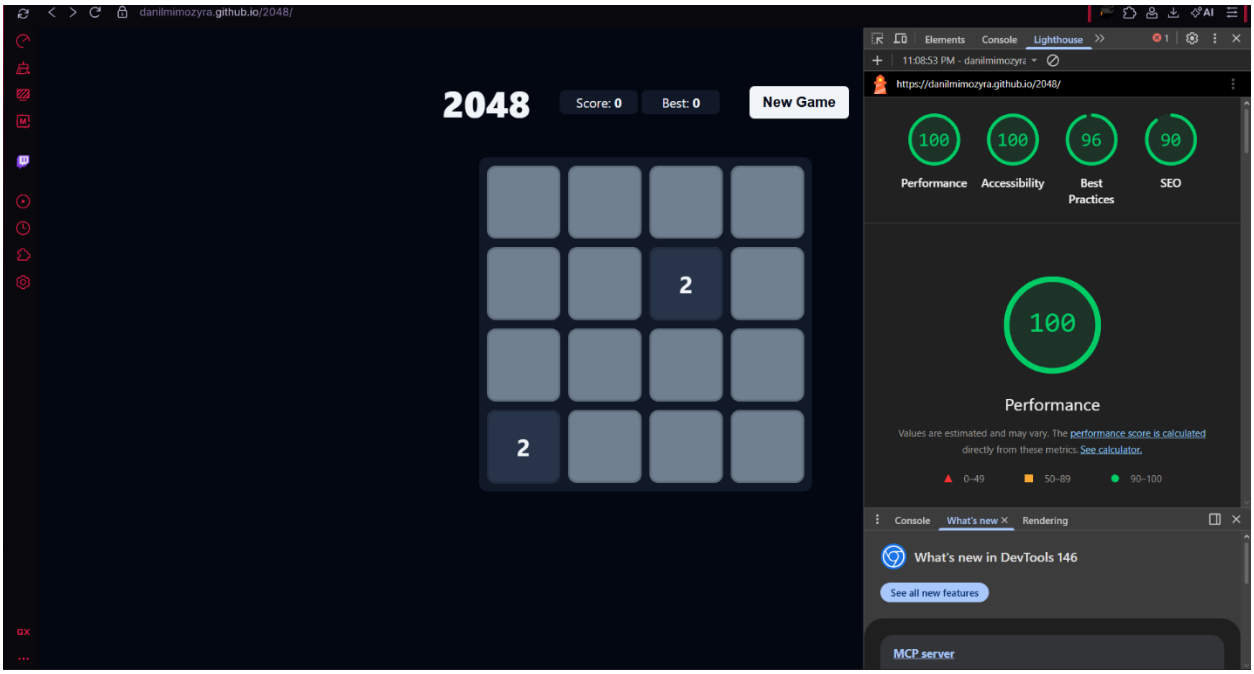
PWA

The application is implemented as a Progressive Web App using a service worker generated by a Vite plugin.

Service Worker Strategy

The service worker uses a caching strategy where application files (HTML, CSS, JavaScript, and assets) are cached during installation. This ensures that the application loads quickly and can work without an internet connection.

Lighthouse Audit



Testing

The application was tested manually by running different scenarios to verify correct behavior of the game logic and user interaction.

Scenario 1 – Game start

Input: New game has started.

Steps: Open the application.

Expected: Two tiles appear on random positions on the board.

Result: Matches expectation.

Scenario 2 – Valid move

Input: Board with multiple tiles.

Steps: Player presses an arrow key (e.g. ArrowRight).

Expected: Tiles move in the selected direction and merge if possible.

Result: Matches expectation.

Scenario 3 – Invalid move (no change)

Input: Board where no tiles can move in a certain direction.

Steps: Player presses a key that does not result in any movement.

Expected: Board remains unchanged, no new tile is spawned.

Result: Matches expectation.

Scenario 4 – Tile merging

Input: Two tiles with the same value next to each other.

Steps: Player moves tiles in their direction.

Expected: Tiles merge into one with a higher value.

Result: Matches expectation.

Scenario 5 – New game button

Input: Game in progress.

Steps: Player clicks "New Game".

Expected: Board resets and score returns to zero.

Result: Matches expectation.

Scenario 6 – Swipe input (mobile)

Input: Game opened on a mobile device.

Steps: Player performs a swipe gesture.

Expected: Tiles move in the direction of the swipe.

Result: Matches expectation.

Scenario 7 – Offline mode

Input: Application loaded and then network is disconnected.

Steps: Turn off internet connection and continue playing.

Expected: Game continues to work normally.

Result: Matches expectation.

Scenario 8 – Edge case (full board)

Input: Board is full and no moves are possible.

Steps: Player attempts to move.

Expected: No tiles move, game remains in the same state.

Result: Matches expectation.

Work Distribution

The work on the project was divided between two main roles:

UI (User Interface): Responsible for the visual design, layout, PWA, responsive behavior, theming, and overall user experience.

Game Logic: Responsible for implementing the core game mechanics, including tile movement, merging, animations, and state management.

Both parts were developed in parallel and then integrated into a single application.

Project Evaluation

What went well

The main functionality of the game works correctly, including tile movement and animations. The application is responsive and supports light and dark themes. PWA support was also successfully implemented.

What was challenging

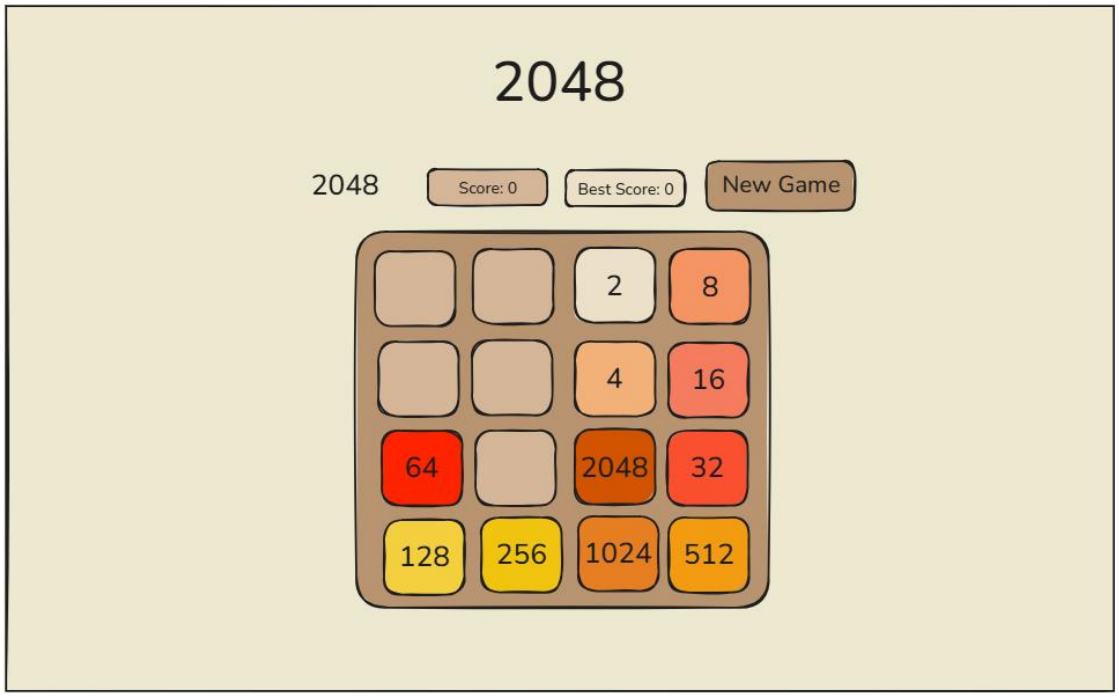
Some features were not fully completed, such as score calculation and win/lose detection. There were also issues with mobile controls, where swipe input does not always work correctly.

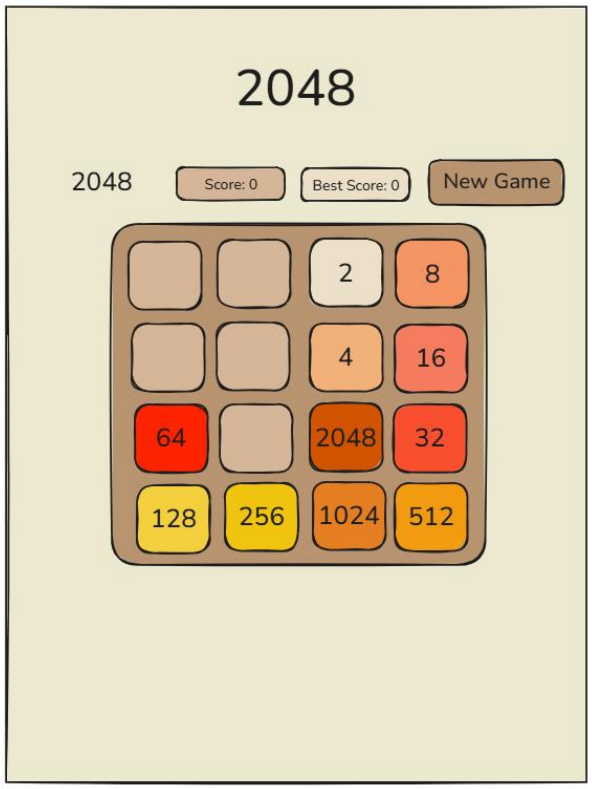
Future improvements

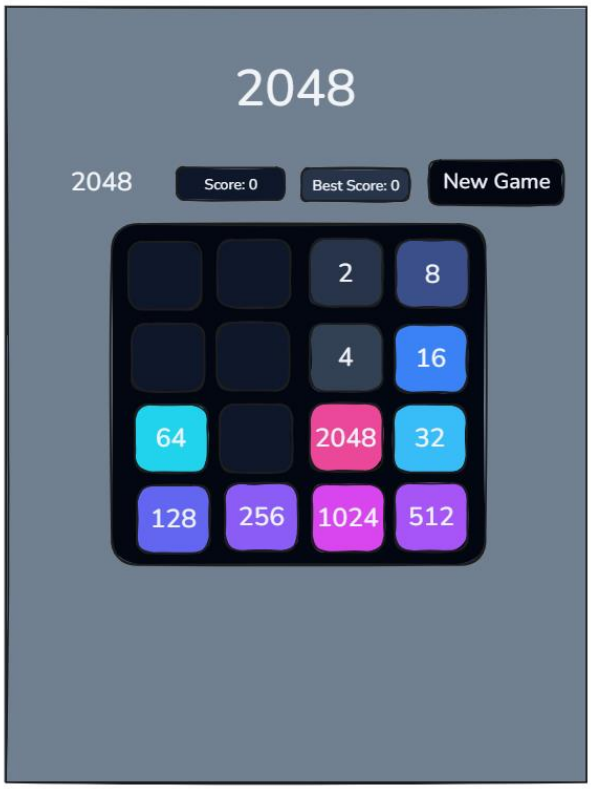
Possible improvements include fixing the score system, adding win/lose detection, improving mobile controls, and saving game progress.

Attachments

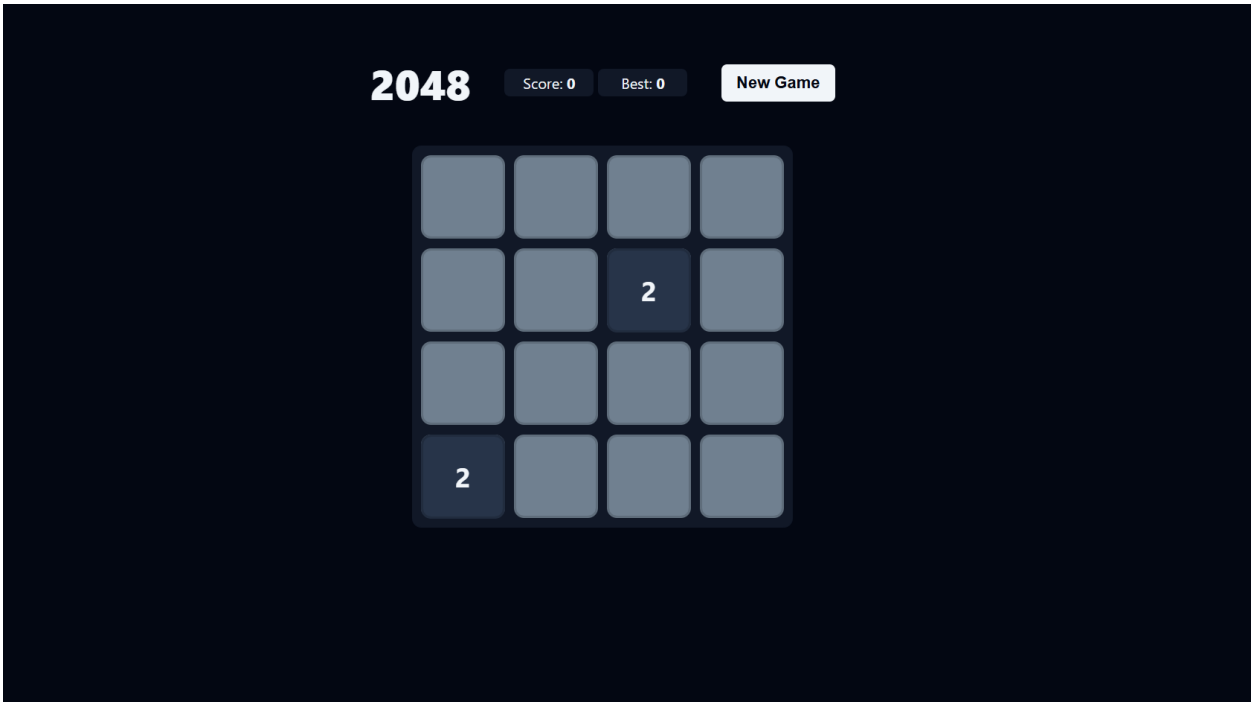
Wireframe (initial design):

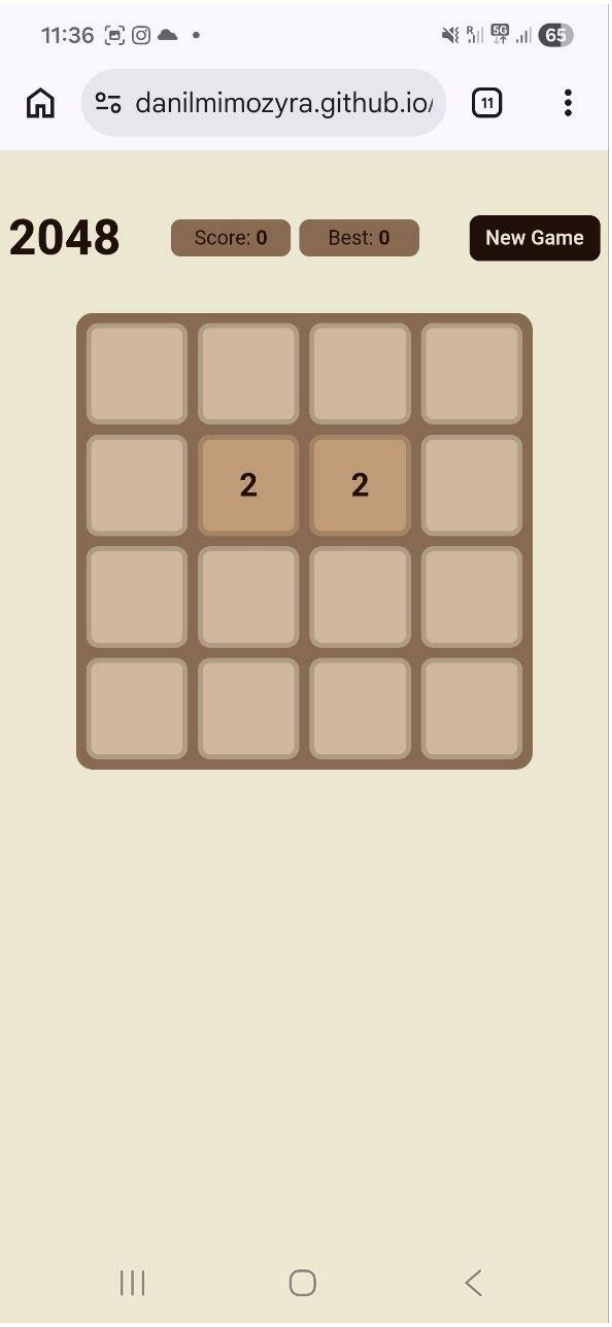
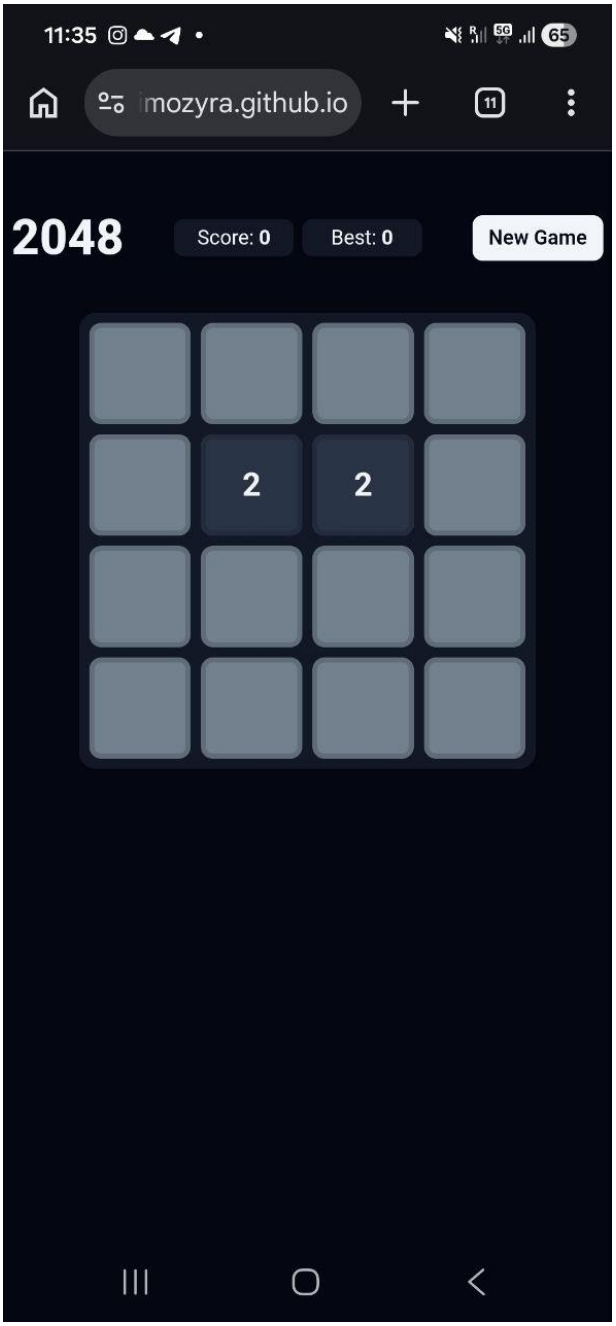






Final application – desktop view:



2048
Score: 0
Best: 0
New Game
2
2

Final application – mobile view: